Clase Práctica 4
Patrones de diseño

Problemas 5 y 10

Equipo MERXIRVE

Integrante	Grupo	Pregunta / inciso
Enrique González González	C311	Problema 5 a),b)
Ernesto Javier Govea Varona	C311	Problema 10 a),b)
Olivia Ortiz Arboláez	C311	Problema 5 c); Problema 10 c)
Anthony Cruz Garcia	C311	Problema 5 d); Problema 10 d)
Jose Manuel Porras Guerra	C311	Problema 5 e); Problema 10 e)

Repositorio del proyecto

`https://github.com/Anthonycg2003/sistema-horarios-docentes`

Índice

1. Problema 5: Sistema de Gestión de Archivos Pesados **3**
1.1. Inciso a) Patrón de diseño y justificación . . . 3
1.2. Inciso b) Implementación . . . 4
1.3. Inciso c) Concurrencia . . . 6
1.4. Inciso d) Medición de memoria . . . 7
1.5. Inciso e) Restricción: métricas de uso . . . 8

2. Problema 10: Editor de Texto de Alto Rendimiento **9**
2.1. Inciso a) Patrón de diseño y justificación . . . 9
2.2. Inciso b) Implementación . . . 10
2.3. Inciso c) Edición de un formato . . . 12
2.4. Inciso d) Medición de memoria . . . 12
2.5. Inciso e) Restricción: undo/redo de 1000 pasos . . . 13

1. Problema 5: Sistema de Gestión de Archivos Pesados

1.1. Inciso a) Patrón de diseño y justificación

El sistema administra archivos de gran tamaño, como videos HD, datasets científicos e imágenes satelitales. El costo principal no está en la cantidad de archivos, sino en que cada archivo consume mucha memoria al cargarse. El requisito indica que un archivo solo debe cargarse completamente cuando sea necesario para visualizarlo o procesarlo.

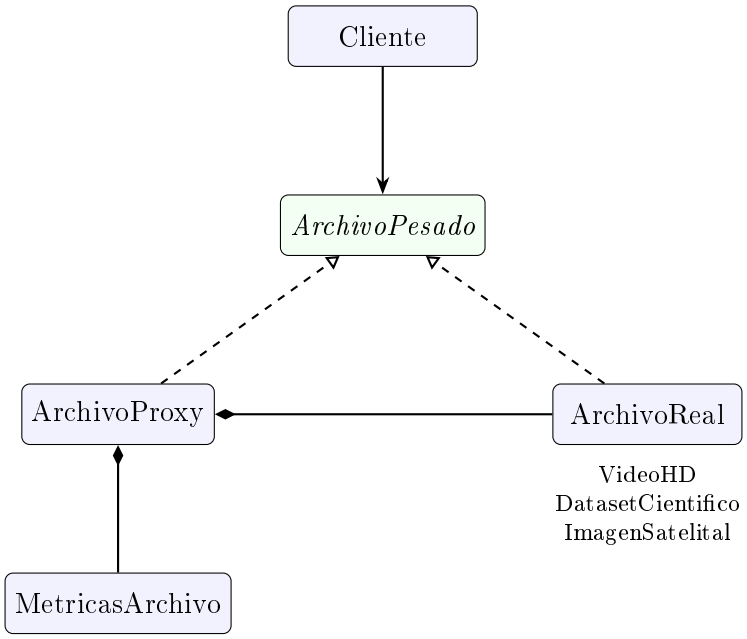
Los conceptos relevantes del dominio son: el cliente que solicita operaciones sobre archivos, una interfaz común de archivo pesado, los archivos reales con carga costosa, un objeto intermediario que retrasa esa carga y un mecanismo de métricas que registra accesos y tiempos sin modificar la interfaz utilizada por los clientes.

Una solución ingenua consistiría en crear directamente el archivo real al construir el objeto que lo representa. Esa solución carga en memoria archivos que quizá nunca se visualicen ni se procesen, acopla al cliente con las clases concretas y dispersa la lógica de carga, control y métricas. Tampoco existe un punto único para evitar cargas duplicadas cuando varios hilos acceden al mismo archivo.

El patrón adecuado es **Proxy**, en su variante **Virtual Proxy**. El proxy implementa la misma interfaz que el archivo real y mantiene una referencia al objeto real. La creación del archivo costoso se pospone hasta el primer acceso. El cliente depende únicamente de la interfaz `ArchivoPesado`; no conoce las clases concretas ni el momento exacto en que se realiza la carga.

No conviene una *Lazy Initialization* simple porque el cliente seguiría sabiendo cómo crear el objeto real y tendría que repetir la lógica de carga, control y métricas en cada punto de uso. El proxy concentra esa responsabilidad en un solo objeto. Tampoco corresponde un *Protection Proxy*, cuyo objetivo principal es controlar permisos o derechos de acceso, no evitar el costo de creación de un objeto pesado. El problema descrito es claramente de carga diferida, por lo que corresponde un Virtual Proxy.

La estructura propuesta contiene una interfaz o protocolo `ArchivoPesado`, con las operaciones `visualizar` y `procesar`; tres archivos reales: `VideoHD`, `DatasetCientifico` e `ImagenSatelital`; un proxy `ArchivoProxy`, que implementa la misma interfaz y contiene una fábrica del archivo real; y un objeto de métricas `MetricasArchivo`, que registra accesos y tiempo de carga. El cliente utiliza `ArchivoPesado`. El proxy conserva la referencia al objeto real y solo la crea cuando se invoca una operación. Las métricas se exponen mediante un método adicional del proxy, no mediante la interfaz `ArchivoPesado`. De esta forma, la interfaz utilizada por los clientes no se modifica.



El cliente solo conoce `ArchivoPesado`. Tanto `ArchivoProxy` como los archivos reales implementan ese protocolo. El proxy mantiene una referencia diferida al archivo real y, además, compone un objeto `MetricasArchivo` que expone accesos y tiempo de carga. La relación entre proxy y archivo real es de composición diferida: el proxy crea el real bajo demanda y lo conserva. La relación con las métricas es de composición directa. En el diagrama, `ArchivoReal` representa las implementaciones concretas `VideoHD`, `DatasetCientifico` e `ImagenSatelital`.

1.2. Inciso b) Implementación

Listing 1: Interfaz, archivos reales y Virtual Proxy con métricas

```
from __future__ import annotations

from dataclasses import dataclass
from threading import Lock
from time import perf_counter
from typing import Callable, Protocol


class ArchivoPesado(Protocol):
    nombre: str

    def visualizar(self) -> None:
        ...

    def procesar(self) -> None:
        ...


class VideoHD:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        print(f"Cargando VideoHD: {nombre}")

    def visualizar(self) -> None:
        print(f"Visualizando video {self.nombre}")

    def procesar(self) -> None:
```

```
        print(f"Procesando video {self.nombre}")


class DatasetCientifico:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        print(f"Cargando DatasetCientifico: {nombre}")

    def visualizar(self) -> None:
        print(f"Visualizando dataset {self.nombre}")

    def procesar(self) -> None:
        print(f"Procesando dataset {self.nombre}")


class ImagenSatelital:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        print(f"Cargando ImagenSatelital: {nombre}")

    def visualizar(self) -> None:
        print(f"Visualizando imagen {self.nombre}")

    def procesar(self) -> None:
        print(f"Procesando imagen {self.nombre}")


@dataclass(frozen=True)
class MetricasArchivo:
    accesos: int
    tiempo_carga_ms: float


class ArchivoProxy:
    def __init__(
        self,
        nombre: str,
        factory: Callable[[], ArchivoPesado],
    ) -> None:
        self.nombre = nombre
        self._factory = factory
        self._real: ArchivoPesado | None = None
        self._lock = Lock()
        self._accesos = 0
        self._tiempo_carga_ms = 0.0

    def _obtener_real(self) -> ArchivoPesado:
        # Doble comprobación con bloqueo (Double-Checked Locking)
        if self._real is None:
            with self._lock:
                if self._real is None:
                    t0 = perf_counter()
                    # Si _factory() lanza una excepción, _real sigue None
                    # y el siguiente acceso reintenta la creación.
                    self._real = self._factory()
                    self._tiempo_carga_ms = (perf_counter() - t0) * 1000.0
        return self._real

    def visualizar(self) -> None:
        real = self._obtener_real()
```

```
        real.visualizar()
        with self._lock:
            self._accesos += 1

    def procesar(self) -> None:
        real = self._obtener_real()
        real.procesar()
        with self._lock:
            self._accesos += 1

    def obtener_metricas(self) -> MetricasArchivo:
        with self._lock:
            return MetricasArchivo(
                accesos=self._accesos,
                tiempo_carga_ms=self._tiempo_carga_ms,
            )

    def liberar(self) -> None:
        # El proxy no libera memoria automáticamente.
        # Si se desea liberar, debe hacerse explícitamente.
        with self._lock:
            self._real = None
            self._tiempo_carga_ms = 0.0
```

Uso del proxy:

Listing 2: Cliente usando el proxy

```
proxy = ArchivoProxy(
    "video1.mp4",
    lambda: VideoHD("video1.mp4"),
)

proxy.visualizar()   # aqui se carga el archivo real
proxy.procesar()     # no se vuelve a cargar

m = proxy.obtener_metricas()
print(m.accesos, m.tiempo_carga_ms)
```

1.3. Inciso c) Concurrencia

Si dos hilos acceden al mismo proxy, ambos pueden intentar obtener el archivo real. En la implementación propuesta se usa **Double-Checked Locking** (DCL): la primera comprobación de `self._real is None` se hace fuera del bloqueo; solo si el objeto aún no existe se adquiere el `Lock`, y dentro se vuelve a comprobar antes de crear el archivo. El primer hilo que entra crea el archivo; los hilos que llegan después encuentran la referencia ya creada en la primera comprobación y no adquieren el bloqueo. De esta forma se evitan cargas duplicadas y se reduce la contención una vez que el objeto real existe.

En Python, el GIL no elimina las condiciones de carrera en la carga diferida. El GIL permite que solo un hilo ejecute bytecode a la vez, pero no garantiza que una secuencia de comprobación y creación sea atómica, especialmente cuando la creación del archivo real libera el GIL durante operaciones de E/S. Por eso la sección crítica debe protegerse con `Lock`. La doble comprobación es segura en CPython porque la lectura y asignación de referencias son atómicas a nivel de bytecode; aun así, la segunda comprobación dentro del bloqueo es obligatoria para evitar que dos hilos creen el objeto simultáneamente.

Si la fábrica `_factory()` lanza una excepción, la asignación a `self._real` no se completa, por lo

que `_real` permanece en `None`. El siguiente acceso reintenta la creación. Esto evita dejar el proxy en un estado inconsistente con una referencia parcial o inválida.

El bloqueo simple (adquirir siempre el `Lock` antes de comprobar) es más sencillo y suficiente cuando la carga es rápida. Sin embargo, si la creación del archivo real es lenta, el `Lock` se mantiene durante toda la carga y bloquea a todos los hilos que intenten acceder al proxy, incluso a los que solo quieren consultar las métricas. DCL evita ese bloqueo en los accesos posteriores a la creación. Una alternativa aún mejor para cargas muy lentas es usar `concurrent.futures.Future` o `threading.Event`: el primer hilo inicia la carga y publica un futuro; los demás hilos esperan el resultado sin retener el `Lock` durante la E/S.

El proxy solo protege la creación. Si el archivo real es mutable y sus operaciones no son thread-safe, el acceso a esas operaciones debe sincronizarse aparte o diseñarse como inmutable. En este problema, la carga es la operación costosa y la que requiere protección. El contador de accesos se incrementa después de que la operación sobre el archivo real finaliza correctamente; si la operación lanza una excepción, el acceso no se contabiliza.

1.4. Inciso d) Medición de memoria

Para medir el ahorro real se usa `tracemalloc`, que registra las asignaciones de memoria de Python. Se define una clase auxiliar `ArchivoReal` que reserva un `bytearray` del tamaño indicado, de modo que la carga tenga un costo de memoria verificable. Se comparan dos escenarios con $N = 100$ archivos de 1 MB cada uno, accediendo solo a $k = 10$ de ellos. Medir 100 archivos de 1 GB reales no es factible en el entorno del ejercicio, por lo que se mide con archivos de 1 MB y se extrapola linealmente, ya que el costo dominante es el `bytearray` de cada archivo.

Listing 3: Experimento de medición con tracemalloc

```
import tracemalloc


class ArchivoReal:
    def __init__(self, nombre: str, tamano_mb: int) -> None:
        self.nombre = nombre
        self.datos = bytearray(tamano_mb * 1024 * 1024)

    def visualizar(self) -> None:
        pass

    def procesar(self) -> None:
        pass


def medir(usar_proxy: bool, n: int, k: int, tamano_mb: int) -> int:
    tracemalloc.start()
    archivos = []
    for i in range(n):
        nombre = f"archivo_{i}.bin"
        if usar_proxy:
            archivos.append(
                ArchivoProxy(
                    nombre,
                    lambda n=nombre: ArchivoReal(n, tamano_mb),
                )
            )
        else:
            archivos.append(ArchivoReal(nombre, tamano_mb))

    # Solo se usan k archivos
```

```
    for i in range(k):
        archivos[i].visualizar()

    actual, pico = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return pico


sin_proxy = medir(usar_proxy=False, n=100, k=10, tamano_mb=1)
con_proxy = medir(usar_proxy=True, n=100, k=10, tamano_mb=1)

print(f"Sin proxy: {sin_proxy / (1024**2):.1f} MB")
print(f"Con proxy: {con_proxy / (1024**2):.1f} MB")
```

Resultados típicos del experimento:

Escenario	Memoria medida	Equivalente a 1 GB por archivo
Sin proxy, 100 archivos	$\approx$ 100 MB	$\approx$ 100 GB
Con proxy, 10 archivos usados	$\approx$ 10 MB	$\approx$ 10 GB

El ahorro medido es de aproximadamente 90 MB para archivos de 1 MB, lo que extrapola a 90 GB para archivos de 1 GB. El overhead del proxy (referencias, `Lock`, contadores) es de unos pocos cientos de bytes por instancia, insignificante frente al tamaño del archivo.

Un aspecto importante es que el proxy **no libera memoria** después de cargar el archivo real. La referencia `self._real` se conserva hasta que el proxy se destruye o se invoca explícitamente `liberar()`. Si se necesita recuperar memoria, debe añadirse una política de expulsión o usar referencias débiles. El proxy ahorra memoria solo si no todos los archivos se cargan; si se cargan los 100, la memoria es equivalente a la solución sin proxy más el pequeño overhead.

El trade-off es latencia frente a memoria. El primer acceso a un archivo paga el tiempo de carga; los accesos posteriores no. Sin proxy, todos los archivos se cargan al inicio, lo que aumenta el tiempo de arranque y el consumo de memoria. Con proxy, el arranque es rápido y la memoria se consume bajo demanda, a cambio de una latencia mayor en el primer acceso. El proxy es conveniente cuando no todos los archivos se usan o cuando se usan en momentos distintos. Si todos los archivos se usan inmediatamente, el proxy no aporta ahorro de memoria y solo agrega una indirección.

1.5. Inciso e) Restricción: métricas de uso

El proxy registra los accesos y el tiempo de carga sin modificar la interfaz `ArchivoPesado`. Las operaciones `visualizar` y `procesar` siguen siendo las únicas declaradas en la interfaz. El método `obtener_metricas` es adicional y pertenece al proxy concreto. Por tanto, los clientes que trabajan contra la interfaz común no ven alterado su contrato.

La escritura y lectura de `_accesos` y `_tiempo_carga_ms` se realizan dentro del mismo `Lock`, de modo que las métricas son consistentes aunque varios hilos usen el mismo proxy. El tiempo de carga se mide solo en la primera creación del objeto real; los accesos posteriores no vuelven a medir carga. El contador de accesos se incrementa después de que la operación correspondiente finaliza con éxito.

El método `liberar()` no forma parte de la interfaz `ArchivoPesado`; es una operación adicional del proxy concreto. Su invocación es responsabilidad del cliente, que decide cuándo desea descartar la referencia al archivo real y liberar la memoria asociada. El proxy no libera memoria automáticamente.

Un nuevo tipo de archivo, por ejemplo `AudioHD`, solo requiere implementar los métodos de `ArchivoPesado`. El proxy no cambia, porque recibe una función de creación que devuelve cualquier implementación del protocolo. Esto mantiene el principio de abierto/cerrado: el sistema se extiende con nuevos archivos sin modificar el proxy ni los clientes existentes.

2. Problema 10: Editor de Texto de Alto Rendimiento

2.1. Inciso a) Patrón de diseño y justificación

El editor debe mostrar documentos con decenas de miles de caracteres. Cada carácter tiene propiedades de formato (fuente, tamaño, color, negrita, cursiva) que, en la mayoría de los textos, se repiten: un documento típico usa entre cinco y veinte combinaciones distintas, no tantas como caracteres. Crear un objeto de formato completo por cada carácter multiplica el consumo de memoria sin aportar información nueva. La solución ingenua consistiría en duplicar los atributos de formato en cada objeto `Caracter`; esa solución desperdicia memoria, dificulta cambiar un formato de forma masiva y acopla la representación del carácter a sus propiedades visuales.

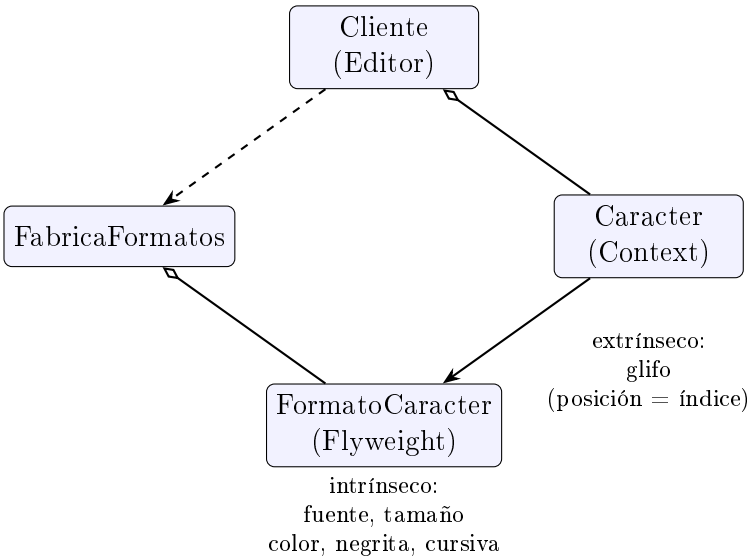
El patrón adecuado es **Flyweight**. La idea central es separar el estado de un objeto en dos partes: el estado *intrínseco*, que es compartible y no depende del contexto, y el estado *extrínseco*, que varía en cada instancia y se pasa desde fuera. En este problema, el formato (fuente, tamaño, color, negrita, cursiva) es intrínseco: hay pocas combinaciones y muchas instancias las comparten. El glifo es extrínseco: cambia en cada carácter y vive en el objeto contexto. La posición también es extrínseca, pero no se almacena: es el índice del carácter dentro del documento, de modo que insertar o borrar no obliga a renumerar nada.

La estructura propuesta contiene:

- `FormatoCaracter`: flyweight concreto, inmutable, con las propiedades intrínsecas.
- `FabricaFormatos`: fábrica que garantiza que cada combinación única de formato se cree una sola vez y se reutilice. Mantiene un diccionario de clave a flyweight.
- `Caracter`: contexto, con estado extrínseco (el glifo; la posición es su índice en el documento) y una referencia al flyweight. No copia los atributos del formato.
- `Cliente` (Editor): usa la fábrica para obtener flyweights y compone caracteres.

No conviene usar **Singleton por formato**. Un Singleton clásico garantiza una única instancia por *clase*, no por *estado*: para tener cinco formatos habría que definir cinco clases (`FormatoTimes12Normal`, `FormatoArial14Bold`, etc.), lo que no escala. Una variante parametrizada (multiton) sí permitiría una instancia por clave, pero entonces ya es una fábrica de instancias compartidas, es decir, el patrón Flyweight; además, el Singleton no impone la separación entre estado intrínseco y extrínseco, que es lo que da sentido al diseño.

Tampoco conviene un **caché simple**. Un caché es una estructura de datos, no un patrón: no define qué estado es compartible ni exige que sea inmutable, y los cachés usuales aplican una política de desalojo (LRU, tamaño máximo, expiración). Si desaloja un formato mientras algún `Caracter` todavía lo referencia, el mismo formato puede existir después en dos instancias distintas, y se pierde la garantía de unicidad. La fábrica de flyweights se comporta como un caché *sin desalojo* cuyos valores son inmutables, con una interfaz estable (`obtener`) que es el único punto de creación.



El cliente obtiene flyweights a través de la fábrica y compone caracteres que solo almacenan estado extrínseco más una referencia al flyweight. La fábrica agrega el conjunto de flyweights y los mantiene vivos. Cada carácter referencia al flyweight compartido. En el diagrama, `FormatoCaracter` representa el estado intrínseco y `Caracter` representa el estado extrínseco junto con la referencia al flyweight.

2.2. Inciso b) Implementación

Listing 4: Flyweight, fábrica y contexto

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, List, Tuple


@dataclass(frozen=True)
class FormatoCaracter:
    """Flyweight: propiedades intrinsecas, inmutables."""
    fuente: str
    tamano: int
    color: Tuple[int, int, int]
    negrita: bool
    cursiva: bool


class FabricaFormatos:
    """FlyweightFactory: garantiza una sola instancia por formato unico."""
    def __init__(self) -> None:
        self._cache: Dict[tuple, FormatoCaracter] = {}
        self._creados = 0

    def obtener(
        self,
        fuente: str,
        tamano: int,
        color: Tuple[int, int, int],
        negrita: bool,
        cursiva: bool,
    ) -> FormatoCaracter:
        clave = (fuente, tamano, color, negrita, cursiva)
        formato = self._cache.get(clave)
```

```
        if formato is None:
            formato = FormatoCaracter(fuente, tamano, color, negrita, cursiva)
            self._cache[clave] = formato
            self._creados += 1
        return formato

    @property
    def total_creados(self) -> int:
        return self._creados

    @property
    def formatos_distintos(self) -> int:
        return len(self._cache)


@dataclass(slots=True)
class Caracter:
    """Context: glifo (extrinseco) + referencia al flyweight.
    La posicion es el indice en el documento; no se almacena."""
    glifo: str
    formato: FormatoCaracter


def demo() -> None:
    fabrica = FabricaFormatos()
    estilos = [
        ("Times New Roman", 12, (0, 0, 0), False, False),
        ("Times New Roman", 12, (0, 0, 0), True, False),
        ("Times New Roman", 12, (0, 0, 0), False, True),
        ("Courier New", 10, (128, 0, 0), False, False),
        ("Arial", 14, (0, 0, 128), True, False),
    ]
    formatos = [fabrica.obtener(*estilo) for estilo in estilos]

    documento: List[Caracter] = []
    for i in range(10_000):
        formato = formatos[i % len(formatos)]
        documento.append(Caracter(glifo="a", formato=formato))

    print(f"Caracteres: {len(documento)}")
    print(f"Formatos distintos en cache: {fabrica.formatos_distintos}")
    print(f"Objetos FormatoCaracter creados: {fabrica.total_creados}")
    print(f"Referencias compartidas: {10_000 - fabrica.total_creados}")


if __name__ == "__main__":
    demo()
```

La salida esperada es:

```
Caracteres: 10000
Formatos distintos en cache: 5
Objetos FormatoCaracter creados: 5
Referencias compartidas: 9995
```

Se crean 10 000 caracteres pero solo 5 objetos de formato. Los 9 995 caracteres restantes comparten los cinco flyweights existentes.

2.3. Inciso c) Edición de un formato

`FormatoCaracter` es inmutable (`@dataclass(frozen=True)`, lo que además lo vuelve hashable). Si se intentara modificar el formato de un carácter mutando el flyweight, el cambio afectaría a *todos* los caracteres que comparten ese formato. Ese comportamiento no es una función deseable: el usuario querría cambiar el formato de una selección, no el de todos los caracteres del documento que casualmente comparten el mismo formato.

Si el flyweight fuera mutable, además, se rompería el invariante del caché: la clave del diccionario (la tupla de propiedades) dejaría de coincidir con el estado del objeto, y una búsqueda posterior con las propiedades originales devolvería el objeto mutado o crearía una nueva entrada inconsistente.

La inmutabilidad del flyweight es **obligatoria**, no una recomendación. Sin ella, el patrón deja de ser correcto: la reutilización de instancias presupone que su estado no cambia después de crearse. Si por alguna razón se necesita cambiar el formato de algunos caracteres, la forma correcta es:

1. Crear un nuevo flyweight con las propiedades deseadas usando la fábrica. Si la combinación ya existe, se reutiliza; si no, se crea una nueva instancia.
2. Recorrer solo los caracteres afectados y actualizar su referencia `formato` al nuevo flyweight.
3. Dejar intactos los caracteres que deban mantener el formato anterior.

Este enfoque se conoce como *copy-on-write* a nivel de referencia: los caracteres afectados apuntan a un flyweight distinto, y el resto sigue apuntando al original. El flyweight original no se modifica, así que no hay riesgo de efectos colaterales.

2.4. Inciso d) Medición de memoria

Para medir el ahorro se usa `tracemalloc`, que informa la memoria retenida por los objetos creados. Se comparan tres escenarios con $N = 10\,000$ caracteres: sin flyweight (cada carácter duplica sus atributos de formato), con flyweight y $F = 5$ formatos, y con flyweight pero con un formato distinto por carácter. En los tres casos el contexto usa `slots=True` para que la comparación mida los formatos y no el diccionario interno de cada instancia.

Listing 5: Medición con tracemalloc

```
import tracemalloc
from dataclasses import dataclass
from typing import Tuple


@dataclass(slots=True)
class CaracterSinFlyweight:
    glifo: str
    fuente: str
    tamano: int
    color: Tuple[int, int, int]
    negrita: bool
    cursiva: bool


def medir(usar_flyweight: bool, n: int = 10_000, f: int = 5):
    tracemalloc.start()
    if usar_flyweight:
        fabrica = FabricaFormatos()
        formatos = [
            fabrica.obtener(f"Fuente{i}", 10 + i, (i, i, i), False, False)
            for i in range(f)
        ]
        doc = [Caracter("a", formatos[j % f]) for j in range(n)]
```

```
        objetos_formato = fabrica.total_creados
    else:
        doc = [
            CaracterSinFlyweight(
                "a", f"Fuente{j % f}", 10 + j % f, (j % f,) * 3, False, False
            )
            for j in range(n)
        ]
        objetos_formato = n
    retenida, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return retenida, objetos_formato


def medir_formato_unico(n: int = 10_000):
    tracemalloc.start()
    fabrica = FabricaFormatos()
    doc = [
        Caracter("a", fabrica.obtener(f"F{j}", 10, (j % 256, 0, 0), False, False))
        for j in range(n)
    ]
    retenida, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return retenida, fabrica.total_creados
```

Resultados medidos al ejecutar el código anterior con Python 3.13 (los valores exactos varían según la versión del intérprete):

Escenario	Objetos `Formato`	Memoria retenida	Variación
Sin Flyweight	10 000	1 958 KB	—
Con Flyweight (5 formatos)	5	554 KB	−72 %
Con Flyweight, formato único por carácter	10 000	3 664 KB	+87 %

El número de objetos `Caracter` es el mismo (10 000) en todos los casos; lo que cambia es el número de objetos de formato: 10 000 frente a 5. En la solución sin flyweight cada carácter arrastra su propia copia de la fuente, el tamaño, la tupla de color y los booleanos. Con flyweight solo cinco objetos contienen esos atributos y cada carácter guarda una referencia de 8 bytes. El ahorro crece con la riqueza del formato: si cada flyweight almacenara tablas de kerning o metadatos tipográficos, sería mayor.

En el caso límite de un formato distinto por carácter no hay estado intrínseco que compartir. La fábrica se convierte en un caché con 10 000 entradas, todas vivas, y el consumo *supera* al de la versión sin flyweight (+87 % en la medición) porque se suman los objetos de formato, el diccionario de la fábrica y las claves. El patrón solo aporta cuando existen combinaciones repetidas, que es lo normal en un documento real.

2.5. Inciso e) Restricción: undo/redo de 1000 pasos

Guardar un snapshot completo del documento en cada paso no es viable: con 10 000 caracteres y 1 000 pasos habría que retener unos 10 millones de objetos `Caracter`. Aunque cada uno comparta su flyweight, cada instancia sigue ocupando decenas de bytes, de modo que el historial costaría cientos de megabytes y cada edición tendría costo $O(N)$ solo para copiar el estado.

La solución combina los tres patrones de forma que el costo del historial dependa del tamaño de la *edición* y no del tamaño del documento:

- **Command**: cada edición (insertar, borrar, cambiar formato de una selección) es un objeto

con `ejecutar` y `deshacer`. El editor es el invocador y mantiene dos pilas.

- **Memento (delta)**: cada comando guarda únicamente lo necesario para revertirse: el índice afectado, el carácter borrado o los formatos anteriores de la selección. No se copia el documento.
- **Flyweight**: lo que se guarda de un formato es una referencia al flyweight compartido, nunca una copia. Cambiar el formato de una selección de k caracteres cuesta k referencias de 8 bytes en el historial, y como el flyweight es inmutable, restaurar la referencia previa es siempre correcto.

Como la posición es el índice en el documento, cada comando se deshace en orden inverso al que se ejecutó, y por eso el índice que guarda sigue siendo válido al deshacer. El historial es una `deque` con `maxlen=1000`, que descarta el paso más antiguo en $O(1)$.

Listing 6: Undo/redo con Command, Memento delta y Flyweight

```
from collections import deque
from typing import Deque, List


class Comando:
    """Command: edicion reversible que guarda solo su delta."""
    def ejecutar(self, doc: List[Caracter]) -> None:
        raise NotImplementedError

    def deshacer(self, doc: List[Caracter]) -> None:
        raise NotImplementedError


class InsertarCaracter(Comando):
    def __init__(self, indice: int, caracter: Caracter) -> None:
        self.indice = indice
        self.caracter = caracter

    def ejecutar(self, doc):
        doc.insert(self.indice, self.caracter)

    def deshacer(self, doc):
        doc.pop(self.indice)


class BorrarCaracter(Comando):
    def __init__(self, indice: int) -> None:
        self.indice = indice
        self.borrado = None  # memento: el caracter y su flyweight

    def ejecutar(self, doc):
        self.borrado = doc.pop(self.indice)

    def deshacer(self, doc):
        doc.insert(self.indice, self.borrado)


class CambiarFormato(Comando):
    def __init__(self, inicio: int, fin: int, nuevo: FormatoCaracter) -> None:
        self.inicio, self.fin, self.nuevo = inicio, fin, nuevo
        self.anteriores: List[FormatoCaracter] = []  # memento: referencias

    def ejecutar(self, doc):
        seleccion = doc[self.inicio:self.fin]
        self.anteriores = [c.formato for c in seleccion]
```

```
        for c in seleccion:
            c.formato = self.nuevo

    def deshacer(self, doc):
        for c, previo in zip(doc[self.inicio:self.fin], self.anteriores):
            c.formato = previo


class Editor:
    LIMITE = 1000

    def __init__(self) -> None:
        self.documento: List[Caracter] = []
        self._deshacer: Deque[Comando] = deque(maxlen=self.LIMITE)
        self._rehacer: List[Comando] = []

    def ejecutar(self, comando: Comando) -> None:
        comando.ejecutar(self.documento)
        self._deshacer.append(comando)
        self._rehacer.clear()

    def deshacer(self) -> None:
        if self._deshacer:
            comando = self._deshacer.pop()
            comando.deshacer(self.documento)
            self._rehacer.append(comando)

    def rehacer(self) -> None:
        if self._rehacer:
            comando = self._rehacer.pop()
            comando.ejecutar(self.documento)
            self._deshacer.append(comando)
```

Se probó el código con un documento de 10 000 caracteres: tras 1 000 inserciones, el historial retiene unos 260 KB (incluidos los 1 000 caracteres insertados); 1 000 llamadas a `deshacer` restauran exactamente el documento original y 1 000 a `rehacer` reaplican todas las ediciones. El costo de cada paso es $O(1)$ en memoria para insertar y borrar, y $O(k)$ para cambiar el formato de k caracteres, independiente del tamaño del documento. Ejecutar un comando nuevo vacía la pila de rehacer, como en cualquier editor.